

 GIT GITHUB VSCODE NETLIFY

De Git a Netlify

Apunte de práctica: cómo publicar tu proyecto paso a paso

Tu compu (VSCode + Git) → GitHub (repositorio online) → Netlify
(proyecto publicado)

1 · ¿Qué es cada herramienta?

Herramienta	¿Para qué sirve?
Git	Programa que controla versiones de tu código (guarda "fotos" de los cambios)
GitHub	Sitio web donde alojamos el repositorio (una copia online del proyecto)
VSCoide	Editor de código donde escribimos y gestionamos todo
Netlify	Servicio que toma tu repo de GitHub y lo publica como página web real

2 · Configuración inicial de Git

Antes de usar Git por primera vez, hay que decirle quién sos. Esto queda guardado de forma **global**: para todos los proyectos que hagas en esa computadora.

Abrí la terminal (podés usar la terminal integrada de VSCoide: `Ctrl + ñ`) y escribí:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tuemail@ejemplo.com"
```

Importante: el mail que pongas acá conviene que sea el mismo con el que te registraste en GitHub, así los commits quedan asociados a tu cuenta.

Para verificar que quedó bien configurado:

```
git config --global --list
```

3 · Crear la carpeta del proyecto y arrancar Git

1. Creá una carpeta para tu proyecto (por ejemplo `mi-proyecto`).
2. Abrí esa carpeta en VSCoide (`Archivo > Abrir Carpeta`).
3. Abrí la terminal integrada y ejecutá:

```
git init
```

Esto convierte esa carpeta en un **repositorio local** de Git. A partir de ahora, Git puede llevar un registro de los cambios que hagas ahí adentro.

4 · El ciclo básico de trabajo con Git

Cada vez que hagas cambios en el proyecto y quieras "guardarlos" en el historial de Git, seguís estos tres pasos:

```
git add .  
git commit -m "Descripción corta de lo que cambié"
```

- `git add .` → le dice a Git "quiero guardar todos los cambios que hice"
- `git commit -m "..."` → crea una "foto" (commit) de ese estado del proyecto, con un mensaje que explica qué se hizo

Tip: los mensajes de commit tienen que ser cortos y claros. Ejemplos: *"Agrego formulario de contacto"*, *"Corrijo estilos del header"*.

5 · Crear el repositorio en GitHub

1. Entrá a **github.com** e iniciá sesión (o creá una cuenta si todavía no tenés).
2. Hacé clic en el botón **New** (o el `+` de arriba a la derecha → *New repository*).
3. Ponele un nombre al repositorio (por ejemplo, el mismo nombre que la carpeta del proyecto).
4. Dejalo en **Public** (así después Netlify puede acceder sin problema).
5. **No marques** ninguna casilla de "Add README", "Add .gitignore" ni licencia.
6. Hacé clic en **Create repository**.

GitHub te va a mostrar una serie de comandos. Vamos a usar los que sirven para conectar un repo que ya existe localmente:

```
git remote add origin https://github.com/tu-usuario/nombre-del-repo.git  
git branch -M main  
git push -u origin main
```

- `git remote add origin ...` → conecta tu repo local con el repo de GitHub
- `git branch -M main` → asegura que la rama principal se llame `main`
- `git push -u origin main` → sube (empuja) tus commits al repositorio en GitHub

La primera vez que hagas `push`, es posible que te pida iniciar sesión en GitHub desde la terminal o el navegador. Seguí las instrucciones que aparezcan en pantalla.

6 · Seguir trabajando y subiendo cambios

Una vez que el repo ya está conectado, el flujo de todos los días es:

```
git add .  
git commit -m "Mensaje del cambio"  
git push
```

Ya no hace falta repetir `git remote add` ni `-u origin main`: alcanza con `git push` a secas.

Podés repetir este ciclo (`add` → `commit` → `push`) cada vez que quieras guardar avances y subirlos a GitHub.

7 · Publicar el proyecto con Netlify

Ahora que el proyecto está en GitHub, vamos a conectarlo con Netlify para que quede online.

1. Entrá a **netlify.com** e iniciá sesión (podés hacerlo directamente con tu cuenta de GitHub, es lo más práctico).
2. Hacé clic en **Add new site** → **Import an existing project**.
3. Elegí **Deploy with GitHub** y autorizá el acceso si te lo pide.
4. Buscá y seleccioná el repositorio que creaste.
5. Netlify va a mostrar las opciones de build:
 - Proyecto **HTML/CSS/JS simple**: dejá "Build command" vacío y como "Publish directory" poné `.` o la carpeta donde está tu `index.html`.
 - Proyecto con **framework** (React, Vite, etc.): Netlify suele detectar automáticamente el build (ej. `npm run build`) y la carpeta de salida (ej. `dist`).
6. Hacé clic en **Deploy site**.

Netlify va a tomar el contenido de tu repo, publicarlo, y darte una URL del estilo:

```
https://nombre-random-1234.netlify.app
```

¡Listo! Tu proyecto ya está online.

8 · Actualizar el sitio publicado

Lo mejor de esta configuración es que **no hay que volver a subir nada manualmente a Netlify**. Una vez conectado el repo:

1. Hacés cambios en tu proyecto localmente.
2. Corrés el ciclo de Git:

```
git add .  
git commit -m "Actualizo la página"  
git push
```

3. Netlify detecta automáticamente el nuevo **push** a GitHub y vuelve a publicar el sitio con los cambios (esto se llama **deploy automático**).

En unos segundos o minutos (según el proyecto), la URL de Netlify ya va a mostrar la versión actualizada.

9 · Resumen del flujo completo

1. `git config --global user.name / user.email` (una sola vez por compu)
2. `git init` (una vez por proyecto)
3. `git add . → git commit -m "..."` (cada vez que guardás avances)
4. Crear repo en GitHub
5. `git remote add origin ... → git push -u origin main` (una vez, para conectar)
6. `git add . → git commit -m "... → git push` (de ahí en más, para subir cambios)
7. Conectar el repo con Netlify (una sola vez)
8. Cada `git push` actualiza automáticamente el sitio publicado

10 · Glosario rápido

Repositorio (repo): la carpeta de tu proyecto, pero con el historial de cambios que registra Git.

Commit: una "foto" guardada de cómo estaba el proyecto en un momento determinado.

Push: subir los commits desde tu compu hacia GitHub.

Remote: la dirección del repositorio en GitHub que está conectada con tu repo local.

Deploy: el proceso de publicar el proyecto para que sea accesible desde una URL en internet.

Build: el proceso de "armar" la versión final del sitio antes de publicarlo (necesario en proyectos con frameworks).

Cualquier error o mensaje raro en la terminal, ¡no entrar en pánico! Léelo con calma, casi siempre indica exactamente qué pasó y en qué archivo/línea.